

LAPORAN TUGAS BESAR
IF1221 - LOGIKA KOMPUTASIONAL
UNI



Disusun Oleh
Kelompok G-25
– ZeriUniDisTriz –

1. Theresia Estelina Ratu Udju (13525088)
2. Edward Terrance Lie (13525127)
3. Christabelcyne Costan (13525141)
4. Natan Danuarta Ariel Wicaksana (13525143)

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA -
KOMPUTASI
INSTITUT TEKNOLOGI BANDUNG KAMPUS JATINANGOR
2026

Daftar Isi

Daftar Isi.....	2
BAB I	
Deskripsi Masalah.....	3
1.1 Tujuan.....	3
1.2 Alur Permainan.....	4
1.3 Spesifikasi.....	5
BAB II	
Penjelasan Commands.....	6
2.1 Perintah Inisialisasi dan Sistem.....	6
2.1.1 startGame.....	6
2.1.2 saveGame.....	9
2.1.3 loadGame.....	12
2.1.4 endGame.....	14
2.2 Perintah Aksi Utama.....	15
2.2.1 mainkanKartu(NomorUrut).....	15
2.2.2 ambilKartu.....	21
2.2.3 tantang.....	26
2.2.4 uni(NomorUrutKartuDiTangan).....	30
2.2.5 tangkap>NamaPemain).....	34
2.3 Perintah Aksi Pendukung.....	38
2.3.1 lihatCommand.....	39
2.3.2 lihatKartu.....	46
2.3.3 cekInfo.....	47
2.4 Fitur Bonus.....	49
2.4.1 sembunyikanKartu(NomorUrut).....	49
2.4.2 tampilkanKartu(NomorUrut).....	52
Bab III	
Hasil Eksekusi Program.....	55
3.1 Inisialisasi Permainan.....	55
3.2 Siklus Turn Permainan.....	56
3.3 Fitur Khusus.....	57
3.3.1 Mekanisme Sembunyikan dan Tampilkan Kartu.....	58
3.3.2 Eksekusi Wildcard dan Plus Four (+4).....	59
3.3.3 Pencegahan Penumpukan Hukuman dan Kartu Wild.....	61
3.3.4 Eksekusi Efek Kartu Reverse dan Skip.....	64
3.3.5 Kombinasi Perintah tangkap., uni., dan tantang.....	66
3.4 Save and Load Game.....	69
3.5 Akhir Permainan.....	70
Lampiran.....	72
- Repository Github.....	72

- Pembagian kerja.....	72
- Surat Pernyataan Penggunaan AI.....	76
Daftar Isi.....	2
BAB I Deskripsi Masalah.....	4
1.1 Tujuan.....	4
1.2 Alur Permainan.....	5
1.3 Spesifikasi.....	6
BAB II Penjelasan Commands.....	7
2.1 Perintah Inisialisasi dan Sistem.....	7
2.1.1 startGame.....	7
2.1.2 saveGame.....	10
2.1.3 loadGame.....	13
2.1.4 endGame.....	14
2.2 Perintah Aksi Utama.....	16
2.2.1 mainkanKartu(NomorUrut).....	16
2.2.2 ambilKartu.....	22
2.2.3 tantang.....	26
2.2.4 uni(NomorUrutKartuDiTangan).....	31
2.2.5 tangkap>NamaPemain).....	35
2.3 Perintah Aksi Pendukung.....	39
2.3.1 lihatCommand.....	39
2.3.2 lihatKartu.....	46
2.3.3 cekInfo.....	48
2.4 Fitur Bonus.....	49
2.4.1 sembunyikanKartu(NomorUrut).....	49
2.4.2 tampilkanKartu(NomorUrut).....	52
Bab III Hasil Eksekusi Program.....	55
3.1 Inisialisasi Permainan.....	55
3.2 Siklus Turn Permainan.....	56
3.3 Fitur Khusus.....	57
3.3.1 Mekanisme Sembunyikan dan Tampilkan Kartu.....	57
3.3.2 Eksekusi Wildcard dan Plus Four (+4).....	59
3.3.3 Pencegahan Penumpukan Hukuman dan Kartu Wild.....	61
3.3.4 Eksekusi Efek Kartu Reverse dan Skip.....	64
3.3.5 Kombinasi Perintah tangkap., uni., dan tantang.....	66
3.4 Save and Load Game.....	68
3.5 Akhir Permainan.....	70
Lampiran.....	71
- Repository Github.....	71
- Pembagian kerja.....	71
- Surat Pernyataan Penggunaan AI.....	75

BAB I

Deskripsi Masalah

1.1 Tujuan

Tujuan dari penyelesaian Tugas Besar ini adalah untuk mengimplementasikan berbagai pengetahuan yang telah dipelajari dalam mata kuliah Logika Komputasional IF1221, materi pra praktikum, serta hasil eksplorasi mandiri mengenai bahasa pemrograman Prolog yang dikerjakan secara berkelompok.

Implementasi permainan kartu "UNI" ini dibangun untuk memenuhi dan mendemonstrasikan pemahaman kami terhadap enam konsep dalam logika komputasional, yaitu:

- **Rekurens:** Diterapkan pada fungsi fungsi pembantu (helper) seperti pencarian elemen dalam *list*, perhitungan panjang *list* tanpa menggunakan fungsi *native*, serta iterasi pemrosesan *save/load* kartu.
- **List:** Diterapkan sebagai struktur data utama untuk merepresentasikan urutan giliran pemain, kumpulan kartu di tangan pemain (*kartu_tangan*), riwayat pemain yang aman (telah mengatakan UNI), serta tumpukan kartu (*discard_top*).
- **Cut:** Diterapkan untuk mengontrol jalur eksekusi program (mencegah *backtracking* yang tidak perlu), mengoptimalkan pencarian kondisi *True*, serta mencegah program jatuh ke dalam kondisi atau *rule* alternatif setelah kondisi utama terpenuhi (misalnya pada implementasi fitur **tangkap** dan pemilihan efek **kartuaksi**).
- **Fail:** Diterapkan sebagai bentuk *error handling* dan pembatalan aksi. Penggunaan **fail** memastikan program menolak aksi ilegal (seperti melempar kartu dengan warna yang tidak sesuai atau menggunakan aksi di luar giliran) tanpa membuat program berhenti (*crash*).

- **Loop:** Diterapkan melalui kombinasi logika rekursif untuk menjalankan alur permainan secara terus-menerus, memproses daftar pemain, dan memanipulasi *state* kartu hingga kondisi kemenangan (`endGame`) tercapai.
- **File Processing:** Diterapkan secara mandiri melalui manipulasi kode ASCII dan pengubahan bentuk *compound term* untuk mendukung fitur penyimpanan kondisi permainan (`saveGame`) dan pemuatan kembali kondisi permainan (`loadGame`) ke dalam *file* berekstensi `.txt`.

1.2 Alur Permainan

- **Inisialisasi (`startGame`):** Permainan dimulai dengan meminta masukan jumlah pemain (2-4 orang) dan nama unik untuk setiap pemain. Urutan giliran kemudian diacak, setiap pemain dibagikan 7 kartu awal, dan satu kartu angka acak diletakkan di tumpukan meja (*discard pile*) sebagai kartu pertama.
- **Fase Giliran:** Pada setiap gilirannya, seorang pemain diperbolehkan melakukan aksi pendukung tanpa batas (seperti `lihatKartu`, `lihatCommand`, atau `cekInfo`). Namun, pemain dibatasi hanya dapat melakukan tepat satu aksi utama (seperti `mainkanKartu`, `ambilKartu`, `tantang`, `tantang`, atau `unl`). Setelah aksi utama dieksekusi, giliran akan otomatis berpindah sesuai arah dan efek permainan.
- **Penyelesaian (`endGame`):** Permainan terus berputar hingga terdapat salah satu pemain yang berhasil menghabiskan seluruh kartu di tangannya. Sistem kemudian akan menghitung poin sisa kartu dari pemain lain, melakukan pengurutan (*sorting*), dan mendeklarasikan peringkat pemenang.
- **Penyimpanan (*State*):** Kapan saja selama permainan berlangsung (di luar proses eksekusi sebuah perintah tertunda), pemain dapat menggunakan fitur `saveGame` untuk menyimpan status permainan saat itu ke dalam *file* `.txt`, dan melanjutkannya di lain waktu dengan perintah `loadGame`.

1.3 Spesifikasi

Sistem permainan dibangun dengan mengelola berbagai fakta dinamis (*dynamic predicates*) untuk mencatat *state* (kondisi) permainan. Spesifikasi aturan dasar yang diterapkan meliputi:

- **Representasi Kartu:** Kartu direpresentasikan dalam bentuk struktur (Compound Term), yaitu `kartu(Warna, Jenis, Status)`.
 - Warna: Merah, Kuning, Hijau, Biru, dan Hitam (untuk *Wildcard*).
 - Jenis: Angka (0-9) dan Aksi (*Skip*, *Reverse*, *Plus Dua*, *Wildcard*, *Plus Empat*).
 - Status: Penanda khusus apakah kartu dalam keadaan *normal* atau *hide* (disembunyikan).
- **Validasi Bermain:** Sebuah kartu hanya valid dibuang ke *discard pile* jika memiliki warna yang sama, jenis/angka yang sama, atau merupakan kartu aksi berwarna Hitam yang bisa dilempar kapan saja.
- **Aturan UNI dan Tangkap:** Pemain yang kartunya tersisa 1 lembar wajib mendeklarasikan kondisinya menggunakan perintah `uni(NomorUrut)`. Jika pemain lalai dan membuang kartu menggunakan perintah biasa, ia kehilangan perlindungan dan dapat di **tangkap** oleh pemain lain, yang berakibat pada penalti pengambilan 2 kartu tambahan. Tuduhan palsu pada fitur tangkap akan berbalik memberikan penalti 1 kartu pada penuduh.
- **Perhitungan Poin Akhir:** Poin dihitung berdasarkan bobot kartu (Kartu angka 1-9 bernilai sesuai angka, kartu 0 bernilai 1 poin, kartu aksi warna bernilai 10 poin, dan kartu hitam bernilai 20 poin). Pemenang utama mendapatkan 0 poin, dan pemain lainnya diurutkan dari poin sisa yang terkecil.

BAB II

Penjelasan Commands

2.1 Perintah Inisialisasi dan Sistem

Kelompok command pada bagian ini bertanggung jawab terhadap keseluruhan manajemen *life-cycle* permainan, mulai dari proses inisiasi state awal, pengelolaan basis fakta dinamis selama run-time berlangsung, mekanisme persistensi state ke media eksternal, hingga terminasi permainan beserta kalkulasi akhir peringkat pemain.

2.1.1 startGame.

Command “startGame.” bertindak sebagai *entry point* utama sistem permainan yang menginisialisasi keseluruhan *state run-time* permainan dan kondisi basis fakta kosong menjadi *state* siap dimainkan. Pada fase ini, sistem melakukan validasi jumlah pemain (`InputJml/1` dan `isJmlPemainValid/1`), registrasi nama unik setiap pemain (`inputPemain/2`), randomize seed random (`randomize/0`), pengacakan urutan giliran (`kocokurutan/4`), distribusi aset kartu awal (`ambilkan_kartu7/1` dan `simpan_kartu/3`), serta pembentukan konfigurasi permainan awal ke dalam memori.

- Validasi jumlah pemain

Proses validasi jumlah pemain diimplementasikan menggunakan pendekatan *fail-driven recursion* berbasis operator kondisional *if-else* (`-> ;`). Predikat `inputJml/1` menerima masukan pengguna lalu melakukan evaluasi bertingkat terhadap dua kondisi utama, yaitu validitas tipe data dan validitas domain nilai. Input hanya diterima apabila bertipe integer dan berada pada rentang 2 sampai 4.

Apabila salah satu kondisi gagal dipenuhi (input berupa tipe data non-integer atau angka di luar rentang), sistem akan menampilkan pesan error yang spesifik lalu memanggil ulang predikat yang sama secara rekursif hingga

memperoleh input yang valid. Predikat `isJmlPemainValid/1` memanfaatkan operator cut (!) untuk menghentikan eksplorasi cabang solusi alternatif setelah kondisi valid terpenuhi, sehingga mekanisme *backtracking* tidak merambat ke fase inisialisasi berikutnya.

Contoh validasi input jumlah pemain
<pre>?- startGame. Masukkan jumlah pemain: tiga. Input Error (Input Harus berbentuk integer) Masukkan jumlah pemain: 5. Input Error (Input Harus berada di range 2-4) Masukkan jumlah pemain: 3.</pre>

- Registrasi nama pemain

Registrasi pemain dilakukan melalui traversal list dinamis yang menyimpan identitas pemain sementara selama proses input berlangsung. Predikat `searchPemain/2` mengimplementasikan mekanisme eliminasi redundansi nama menggunakan evaluasi rekursif terhadap elemen list.

Saat elemen kepala list berhasil cocok dengan nama input, predikat memanfaatkan fail untuk memaksa evaluasi menghasilkan *false*, sehingga nama dianggap telah digunakan. Sebaliknya, apabila traversal mencapai list kosong tanpa kecocokan, nama dinyatakan valid dan dapat diinstansiasikan ke dalam daftar pemain.

Contoh validasi input nama pemain
<pre>?- startGame. Masukkan jumlah pemain: 3. Masukkan nama pemain 1: 'Upin'. Masukkan nama pemain 2: 'Upin'. Nama sudah digunakan. Masukan nama lain: 'Ipin'. Masukkan nama pemain 3: 123. Input harus berupa tipe atomic dengan kata yang diapit '' Masukkan nama pemain 3: 'Apin'.</pre>

- Pengacakan giliran

Urutan giliran pemain dihasilkan melalui mekanisme random traversal menggunakan predikat `random_select/4`. Pada setiap iterasi, satu elemen pemain dipilih secara acak dari himpunan kandidat aktif, dieliminasi dari list asal, lalu dipindahkan ke ekor list hasil menggunakan `insert_tail/3`.

Proses ini berlangsung secara rekursif hingga seluruh elemen list habis diproses, menghasilkan permutasi urutan pemain yang bersifat acak dan bebas duplikasi. Pendekatan ini membentuk mekanisme shuffle deklaratif tanpa memerlukan manipulasi indeks array.

- Distribusi kartu awal

Distribusi kartu awal dilakukan melalui iterasi traversal terhadap list urutan pemain. Untuk setiap pemain, sistem membangkitkan tujuh buah objek compound term berbentuk `kartu(Warna, Jenis, Status)` melalui predikat `ambilkan_kartu7/1`.

Setelah daftar kartu berhasil dibentuk, predikat `assertz/1` digunakan untuk menginjeksikan ke `kartu_tangan>NamaPemain, DaftarKartu` ke dalam run-time memory GNU Prolog. Penggunaan operator cut (!) setelah proses `assert` bertujuan untuk mencegah re-evaluasi akibat mekanisme *backtracking*, sehingga distribusi kartu yang telah terbentuk tidak mengalami regenerasi otomatis pada fase permainan berikutnya. Setelah pemain memiliki masing-masing kartu. Program akan mengambil 1 kartu random (tidak berjenis wildcard, plus_empat, plus_dua, skip, maupun reverse) menggunakan rule `random_discardpile/1.`, sebagai discard-top atau `discard_pile/1`. Kartu ini akan dijadikan pijakan kartu pertama dalam memandu jalannya permainan.

Skenario <code>startGame</code> yang berhasil
<pre>?- startGame. Masukkan jumlah pemain: 3.</pre>

```
Masukkan nama pemain 1: 'Upin'.
Masukkan nama pemain 2: 'Ipin'.
Masukkan nama pemain 3: 'Apin'.

Upin - Apin - Ipin

Setiap pemain mendapatkan 7 kartu acak

Kartu discard top: merah-7

Giliran Upin

yes
```

2.1.2 saveGame.

Command “**saveGame.**” berfungsi sebagai mekanisme serialisasi yang mentransformasikan keseluruhan basis fakta permainan menjadi file teks berekstensi **.txt**.

Proses eksekusi command ini terikat pada kondisi state awal permainan dan efek kartu yang sedang aktif.

Jika permainan belum diinisialisasi, sistem akan langsung menolak perintah, menampilkan pesan restriksi, dan memicu *fail*.

Skenario 1 (Gagal karena **game_started** masih bernilai false)

```
?- saveGame.
Maaf Fitur ini tidak dapat digunakan jika belum startGame atau loadGame

no
```

Jika permainan sudah dimulai namun sistem mendeteksi adanya efek kartu **plus_dua** atau **plus_empat** yang sedang aktif, proses penyimpanan akan dibatalkan demi menjaga keadilan permainan agar pemain menyelesaikan kewajiban mengambil kartu terlebih dahulu.

Skenario 2 (Gagal karena masih adanya efek kartu `plus_dua/plus_empat`)

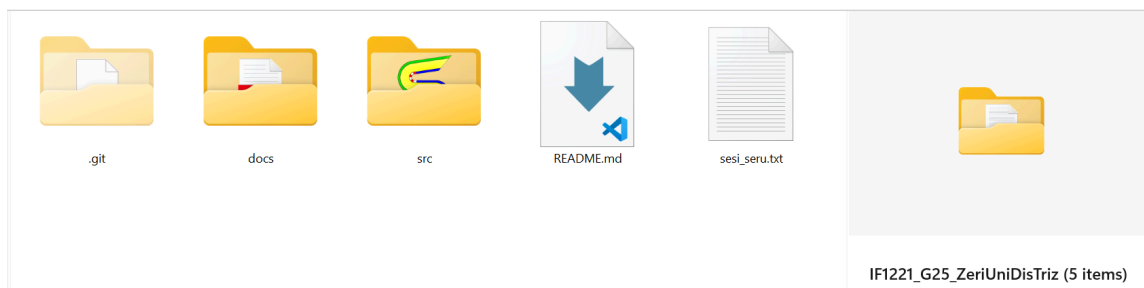
```
?- saveGame.  
Anda tidak dapat saveGame saat giliran ini!  
  
no
```

Jika tidak ditemukan keberadaan kartu efek, sistem melanjutkan proses dengan terlebih dahulu meminta nama file tujuan dari pengguna, kemudian memanfaatkan predikat `insert_txt/2` untuk membentuk nama file lengkap melalui manipulasi list ASCII. Setelah nama file berhasil dibentuk, sistem membuka output stream menggunakan `open(FileName, write, Stream)` untuk memulai proses serialisasi data permainan.

Skenario 3 (Kondisi `saveGame` berhasil)

```
?- saveGame.  
Masukkan nama file penyimpanan: sesi_seru.  
Status permainan berhasil disimpan ke sesi_seru.txt.  
  
yes
```

Sebagai bukti keberhasilan proses eksekusi, sistem akan langsung membentuk file eksternal baru di dalam direktori utama proyek.



Contoh Isi File `.txt` ketika tidak ada kartu yang disembunyikan

```
urutan_pemain: ['Ipin', 'Apin', 'Upin'].  
giliran: 'Ipin'.  
discard_top: kuning-reverse.
```

```

warna_aktif:kuning.
arah_permainan:kiri.
status_UNI:[].
kartu('Ipin'):[biru-5,hijau-5,hitam-wildcard,hijau-1,hijau-9,biru-5,biru-2,hitam-wildcard].
kartu('Apin'):[kuning-plus_dua,merah-0,kuning-6,hijau-4,hijau-7,kuning-0].
kartu('Upin'):[biru-1,hijau-5,hijau-3,hijau-4,biru-7,biru-5,kuning-5].

```

Contoh Isi File .txt ketika ada kartu yang disembunyikan

```

urutan_pemain: ['Ipin','Apin','Upin'].
giliran:'Apin'.
discard_top:kuning-4.
warna_aktif:kuning.
arah_permainan:kanan.
status_UNI:[].
kartu('Ipin'):[hitam-plus_empat,kuning-7-hide,biru-5-hide,kuning-7,biru-skip,kuning-4-hide,kuning-7].
kartu('Apin'):[kuning-1,merah-skip,merah-skip,biru-3,biru-7,biru-skip,hijau-8-hide,kuning-reverse].
kartu('Upin'):[hitam-wildcard-hide,kuning-reverse,hijau-reverse,hitam-wildcard,biru-5,hitam-plus_empat,merah-plus_dua,hitam-plus_empat].

```

Setelah “saveGame.” berhasil dijalankan, program akan melakukan **retratcall** terhadap semua komponen permainan di dalam game sehingga game dianggap telah berhenti. Jika pemain ingin melakukan aksi utama maupun aksi pendukung, pemain harus melakukan “loadGame.” atau “startGame.” terlebih dahulu.

Skenario 4 (Kondisi setelah saveGame berhasil)

```
| ?- saveGame.
Masukkan nama file penyimpanan: sesi_seru.
Status permainan berhasil disimpan ke sesi_seru.txt.

(31 ms) yes
| ?- mainkanKartu(1).
Maaf Fitur ini tidak dapat digunakan jika belum startGame atau loadGame

no
```

2.1.3 loadGame.

Command “loadGame.” merupakan mekanisme deserialisasi state permainan yang bertugas merekonstruksi file permainan eksternal kembali ke dalam basis fakta dinamis GNU Prolog. Proses eksekusi command ini terikat pada kondisi status awal permainan. Sistem terlebih dahulu mengecek status `game_started`.

Jika flag `game_started` sudah aktif/bernilai *true*, sistem akan langsung menolak proses memuat data demi menjaga konsistensi sesi permainan yang sedang berjalan.

Skenario 1 (Gagal karena game lain sudah dimulai)
<pre>?- loadGame. Maaf loadGame hanya dapat dilakukan sebelum game dimulai no</pre>

Jika permainan belum dimulai, sistem akan meminta masukan nama file dari pengguna. Namun, apabila nama file yang dimasukkan tidak ada di dalam direktori penyimpanan, predikat `file_exists/1` akan menghasilkan nilai *false* dan membatalkan proses restorasi.

Skenario 2 (Gagal karena file tidak ditemukan)
<pre>?- loadGame. Masukkan nama file yang akan dimuat: sesi_salah.</pre>

Maaf Nama file yang anda masukkan tidak tersedia

no

Jika file eksternal ditemukan dan dinyatakan valid, sistem akan melanjutkan ke tahap reset *run-time* lama dan rekonstruksi seluruh *state* permainan.

- Reset *run-time* lama

Sebelum proses pembacaan file dimulai, sistem terlebih dahulu melakukan pembersihan *run-time memory* menggunakan `retractall/1` terhadap seluruh fakta dinamis aktif. Tahapan ini bertujuan mencegah terjadinya overlap state maupun inkonsistensi data akibat sisa fakta permainan sebelumnya.

- Rekonstruksi state permainan

File eksternal dibuka dalam mode baca menggunakan `open(FileName, Read, Stream)` lalu dibaca secara bertahap menggunakan `read/2`. Proses rekonstruksi aset kartu pemain dilakukan dengan memanfaatkan predikat rekursif `loadkartu/3` dan `loadkartuhelper/4`.

Setelah objek kartu berhasil direkonstruksi, fakta-fakta tersebut dimasukkan kembali ke *runtime memory* menggunakan `assertz/1` sehingga permainan dapat dilanjutkan tepat pada *state turn* terakhir sebelum proses penyimpanan dilakukan.

Skenario 3 (loadGame berhasil)

?- loadGame.

Masukkan nama file yang akan dimuat: sesi_seru.

Status permainan berhasil dimuat dari sesi_seru.txt.

Melanjutkan Giliran Upin.

yes

2.1.4 endGame.

Command endGame. digunakan untuk menghentikan permainan sekaligus menjalankan proses kalkulasi skor akhir dan pembentukan ranking pemain berdasarkan total poin sisa kartu. Proses eksekusi *command* ini terikat pada kondisi kartu pemain, saat

kartu pemain saat ini telah habis, *command* ini akan dipanggil. Selanjutnya, sistem akan mengambil status pemain yang sedang memegang giliran aktif melalui *giliran(Pemenang)* sebagai basis deklarasi akhir permainan, menghitung seluruh poin kartu secara rekursif, dan menampilkan klasemen akhir.

- Kalkulasi total poin

Sistem mengumpulkan seluruh kartu tersisa dari masing-masing pemain melalui predikat *total_poin_pemain/2* dan *hitung_list_poin/2*. Setiap jenis kartu dipetakan terhadap bobot poin tertentu berdasarkan aturan permainan.

Pemetaan poin dilakukan sebagai berikut:

- Kartu angka bernilai sesuai nominalnya (kecuali 0).
- Kartu angka 0 bernilai 1 poin.
- Kartu aksi (*skip*, *reverse*, *plus_dua*) bernilai 10 poin.
- Kartu wild (*wildcard*, *plus_empat*) bernilai 20 poin.

Proses kalkulasi berlangsung secara rekursif melalui traversal list kartu pemain hingga seluruh elemen selesai dievaluasi.

- Tampilan perhitungan

Untuk menghasilkan tampilan kalkulasi poin yang informatif, sistem memanfaatkan predikat *jabarkan_kartu/4* yang menyusun representasi string penjumlahan poin menggunakan operasi konkatenasi ASCII melalui *sambung_tiga/4*.

- Sorting ranking

Setelah seluruh poin berhasil dihitung, daftar pemain diurutkan menggunakan pendekatan bubble sort deklaratif melalui predikat *sort_pemain/2* dan pembandingan *lebih_baik/2*.

Kriteria pengurutan dilakukan secara hierarkis berdasarkan total poin terkecil, jumlah kartu tersisa paling sedikit, dan posisi giliran pemain pada urutan awal permainan. Permainan dengan total poin yang paling rendah ditempatkan pada peringkat tertinggi sebagai pemenang akhir permainan.

Skenario 1 (endGame berhasil)

Permainan selesai! Apin menghabiskan semua kartunya!

Berikut perhitungan poin sisa kartu.

Upin: merah-7 + hijau-skip = 7 + 10 = 17 poin

Ipin: kuning-2 + hitam-wildcard = 2 + 20 = 22 poin

Apin: kartu habis = 0 poin

Urutan pemenang:

1. Apin (0 poin)
2. Upin (17 poin)
3. Ipin (22 poin)

Selamat, Apin menjadi pemenang!

yes

2.2 Perintah Aksi Utama

Kelompok *command* pada bagian ini berisi aksi inti permainan yang secara langsung memanipulasi *state* permainan selama turn berlangsung. Seluruh *command* pada kategori ini berinteraksi dengan basis fakta dinamis untuk menghasilkan transisi *state* permainan secara deklaratif. Setiap aksi pemain akan memicu evaluasi rule tertentu yang dapat memodifikasi giliran, distribusi kartu, efek kartu spesial, maupun kondisi kemenangan permainan.

2.2.1 mainkanKartu(NomorUrut).

Command “mainkanKartu(NomorUrut).” merupakan *command* yang digunakan pemain untuk melempar satu buah kartu dari tangannya ke atas *discard pile* (meja). Keberhasilan perintah ini sangat bergantung pada validasi kecocokan kartu dan efek yang sedang aktif di arena permainan. Alur logika “mainkanKartu(NomorUrut).” antara lain:

- Validasi Kondisi State Permainan:

Sistem mengecek apakah permainan sudah dimulai (`startGame` atau `loadGame`).

- Validasi Efek Penalti:

Sistem memeriksa fakta “`efek/1`”. Jika pemain saat ini sedang berada di bawah efek `plus_dua` atau `plus_empat`, pemain kehilangan hak untuk memainkan kartu dan *command* akan langsung digagalkan (`fail`).

- Validasi Kepemilikan dan Kecocokan Kartu:

Sistem mengekstrak kartu dari tangan pemain berdasarkan indeks masukan (`ambil_kartu_ke`). Jika indeks valid, sistem mengadu kartu tersebut dengan kartu teratas di meja (`discard_top`) menggunakan *rule* pembantu `cek_validitas/2`. Kartu dinyatakan valid jika memiliki warna atau angka/aksi yang sama, atau merupakan *wildcard*.

- Pencegahan Tumpuk Kartu :

Sistem melarang pemain memainkan kartu `plus_dua` di atas `plus_dua`, serta `plus_empat` di atas `plus_empat` atau *wildcard* secara berturut-turut.

- Pembaruan State dan Transisi Giliran:

Jika seluruh validasi lolos, sistem akan membuang kartu dari `kartu_tangan`, memperbarui `discard_top`, menyimpan memori status kartu sebelumnya (khusus untuk +4 agar bisa dievaluasi oleh *command*antang), mengeksekusi `efek_aksi` dari kartu yang dilempar, lalu menghitung dan memindahkan giliran ke pemain selanjutnya.

Skenario 1 (`mainkanKartu`. dilakukan saat terkena efek hukuman `plus_dua` atau `plus_empat`):

Contoh Hasil Eksekusi Program

```
| ?- mainkanKartu(1).  
Anda tidak dapat mainkanKartu saat giliran ini!  
  
no
```

Skenario 2 (mainkanKartu. dilakukan dengan Nomor Urut yang berada di luar jumlah kartu di tangan):

Contoh Hasil Eksekusi Program
<pre> ?- startGame. Masukkan jumlah pemain: 2. Masukkan nama pemain 1: 'Apin'. Masukkan nama pemain 2: 'Ipin'. Apin - Ipin Setiap pemain mendapatkan 7 kartu acak Kartu discard top: hijau-4 Giliran Apin (31 ms) yes ?- mainkanKartu(10). Nomor urut tidak valid! Membatalkan aksi. no</pre>

Skenario 3 (mainkanKartu. dilakukan tetapi kartu yang dipilih tidak cocok dengan warna/jenis di meja):

Contoh Hasil Eksekusi Program
<pre> ?- lihatKartu.</pre>

Berikut kartu yang anda miliki.

1. kuning-5
2. hitam-plus_empat
3. biru-5
4. biru-9
5. hitam-wildcard
6. merah-5
7. kuning-1

yes

| ?- mainkanKartu(1).

Kartu tidak valid! Warna atau angkanya tidak cocok dengan kartu di meja.

no

Skenario 4 (mainkanKartu. dilakukan untuk menumpuk kartu +2 di atas kartu +2):

Contoh Hasil Eksekusi Program

| ?- cekInfo.

Kartu discard top: merah-plus_dua

Urutan pemain: Ipin, Upin, Apin

Informasi pemain:

- Ipin: 2 kartu
- Upin: 7 kartu
- Apin: 7 kartu

(15 ms) yes

| ?- lihatKartu.

Berikut kartu yang anda miliki.

1. kuning-plus_dua
2. hijau-1

yes

| ?- mainkanKartu(1).

Kartu +2 tidak boleh dimainkan berturut-turut!

no

Skenario 5 (mainkanKartu. dilakukan untuk menumpuk kartu +4 di atas kartu +4)

Contoh Hasil Eksekusi Program

```
| ?- lihatKartu.
```

Berikut kartu yang anda miliki.

1. merah-8
2. hijau-8
3. hitam-wildcard
4. hitam-plus_empat
5. hitam-wildcard
6. kuning-5
7. merah-plus_dua
8. kuning-2

yes

```
| ?- cekInfo.
```

Kartu discard top: hitam-plus_empat

Warna aktif: kuning

Urutan pemain: Upin, Ipin, Apin

Informasi pemain:

- Upin: 12 kartu
- Ipin: 8 kartu
- Apin: 6 kartu

yes

```
| ?- mainkanKartu(4).
```

Kartu +4 tidak boleh dimainkan berturut-turut!

no

Skenario 6 (`mainkanKartu`. dilakukan untuk menumpuk kartu wildcard di atas kartu wildcard)

```
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. hitam-plus_empat  
2. merah-0  
3. biru-3  
4. biru-5  
5. hitam-plus_empat  
6. hitam-wildcard  
7. kuning-skip  
8. merah-8  
9. merah-0  
10. merah-6  
11. hitam-wildcard  
12. hijau-8  
  
(31 ms) yes  
| ?- cekInfo.  
Kartu discard top: hitam-wildcard  
Warna aktif: merah  
Urutan pemain: Upin, Ipin, Apin  
Informasi pemain:  
- Upin: 12 kartu  
- Ipin: 7 kartu  
- Apin: 7 kartu  
  
yes  
| ?- mainkanKartu(11).  
Kartu wildcard tidak boleh dimainkan berturut-turut!  
  
no
```

Skenario 7 (`mainkanKartu`. berhasil dilakukan secara normal dan giliran berpindah):

Contoh Hasil Eksekusi Program

```
| ?- cekInfo.  
Kartu discard top: merah-5  
Urutan pemain: Upin, Ipin, Apin  
Informasi pemain:  
- Upin: 2 kartu  
- Ipin: 7 kartu  
- Apin: 7 kartu  
  
(16 ms) yes  
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. merah-9  
2. hijau-1  
  
yes  
| ?- mainkanKartu(1).  
Upin memainkan kartu: merah-9  
  
--- Giliran Selesai ---  
Giliran Ipin  
  
yes
```

2.2.2 ambilKartu.

Command “ambilKartu.” merupakan command yang digunakan untuk memberikan kartu terhadap pemain saat ini. Penambahan kartu bervariasi sesuai dengan efek yang didapat oleh pengguna. Alur logika “ambilKartu.” antara lain :

- Validasi Kondisi State Permainan

Jika state permainan game dimulai belum terdefinisi, mekanisme **ambilKartu.** tidak dapat digunakan.

Skenario 1 (“ambilKartu.” dilakukan sebelum “startGame.” atau “loadGame.”):

Contoh Hasil Eksekusi Program
<pre> ?- ambilKartu. Maaf Fitur ini tidak dapat digunakan jika belum startGame atau loadGame no</pre>

- Skenario 2 (“ambilKartu.” dilakukan setelah “loadGame.”)

Contoh Hasil Eksekusi Program
<pre> ?- loadGame. Masukkan nama file yang akan dimuat: a. Status permainan berhasil dimuat dari a.txt. Melanjutkan Giliran Apin. (16 ms) yes ?- ambilKartu. Apin mendapatkan kartu: hijau-plus_dua Giliran Ipin yes</pre>

- Validasi Efek

Sistem akan memvalidasi efek yang sedang berjalan saat ini. Jika ditemukan efek “plus_dua”, command “ambilKartu.” akan menjalankan pengambilan kartu sebanyak dua. Jika ditemukan efek “plus_empat”, command “ambilKartu.” akan menjalankan pengambilan kartu sebanyak empat. Selain dari kedua itu, command ini hanya akan menjalankan pengambilan kartu sebanyak satu.

- Pengecekan Status UNI pada giliran pemain saat ini

Selanjutnya, sistem akan memvalidasi apakah pemain berada di dalam (list_uni/1). Jika pemain berada di (list_uni/1), pemain akan di-*remove* dari list tersebut. Jika tidak berada di (list_uni/1), program akan melanjutkan proses

- Penambahan kartu

- Kondisi Normal

Sistem akan memberikan satu kartu secara random ke dalam list kartu pemain dengan giliran saat ini.

- Skenario 3 (“ambilKartu.” dilakukan setelah “startGame.” atau ambilKartu biasa)

Contoh Hasil Eksekusi program
<pre> ?- startGame. Masukkan jumlah pemain: 3. Masukkan nama pemain 1: 'Apin'. Masukkan nama pemain 2: 'Upin'. Masukkan nama pemain 3: 'Ipin'. Apin - Ipin - Upin Setiap pemain mendapatkan 7 kartu acak Kartu discard top: hijau-1 Giliran Apin (47 ms) yes ?- ambilKartu. Apin mendapatkan kartu: hitam-wildcard Giliran Ipin yes</pre>
<pre> ?- mainkanKartu(5). Ipin memainkan kartu: hijau-7 --- Giliran Selesai ---</pre>

Giliran Upin

(15 ms) yes

| ?- ambilKartu.

Upin mendapatkan kartu: hitam-plus_empat

Giliran Apin

yes

- Kondisi **plus_dua** atau **plus_empat**

Sistem akan mengecek efek kartu yang berlaku saat ini. Efek **plus_dua** akan memberikan dua kartu ke dalam listkartu pemain, sedangkan efek **plus_empat** akan memberikan empat kartu ke dalam listkartu pemain.

- Skenario 4 (“ambilKartu.” dilakukan setelah terkena efek **plus_dua**)

Contoh Hasil Eksekusi Program

| ?- mainkanKartu(8).

Apin memainkan kartu: hijau-plus_dua

--- Giliran Selesai ---

Giliran Ipin

yes

| ?- ambilKartu.

Ipin mendapatkan kartu: hitam-wildcard

Ipin mendapatkan kartu: merah-3

Giliran Upin

yes

- Skenario 5 (“ambilKartu.” dilakukan setelah terkena efek plus_empat)

Contoh Hasil Eksekusi Program
<pre> ?- mainkanKartu(4). Upin memainkan kartu: hitam-plus_empat Pilih warna: 1. Merah 2. Kuning 3. Hijau 4. Biru Pilihan anda: 3. Warna aktif sekarang adalah hijau. Kartu +4 dikeluarkan! Pemain berikutnya bisa menantang. --- Giliran Selesai --- Giliran Upin (79 ms) yes ?- ambilKartu. Upin mendapatkan kartu: merah-2 Upin mendapatkan kartu: merah-reverse Upin mendapatkan kartu: kuning-9 Upin mendapatkan kartu: kuning-7 Giliran Apin yes</pre>

2.2.3 tantang.

Command “**tantang.**” digunakan untuk melakukan tantangan terhadap penggunaan kartu wild draw four (**plus_empat**) karena penggunaan wild draw four seharusnya hanya digunakan saat pemain tidak memiliki kartu baik warna maupun jenis yang sama dengan yang berada di (**discard_top/1**) saat ini. Alur command ini adalah sebagai berikut :

- Validasi kondisi state permainan game

Jika state permainan game belum terdefinisi, mekanisme tantang tidak dapat digunakan.

Skenario 1 **tantang** sebelum **startGame** atau **loadGame**

```
| ?- tantang.  
Maaf Fitur ini tidak dapat digunakan jika belum startGame atau loadGame  
  
no
```

- Validasi efek(‘**plus_empat**’)

Jika efek(‘**plus_empat**’) berlaku saat ini maka validasi program berhasil. Jika tidak, program akan mengeluarkan notifikasi error.

Skenario 2 tantang disaat belum ada yang memainkan wild draw four

```
| ?- tantang.  
Tidak bisa menantang saat ini, belum ada orang yang mengeluarkan kartu  
wild draw four (plus_empat)  
  
no
```

- Pengecekan Kartu dan Penambahan Kartu

Pada tahap ini, program akan mengecek kartu pemain yang mengeluarkan **plus_empat** menggunakan term (**cek_kecocokan/3**). Jika pemain yang mengeluarkan **plus_empat** memiliki kartu yang sesuai, pemain tersebut akan mendapatkan 4 kartu tambahan sebagai hukuman. Sebaliknya, jika pemain yang mengeluarkan **plus_empat** memang tidak memiliki kartu yang sesuai, pemain yang menantang akan mendapatkan 6 kartu secara acak.

Skenario 3 Tantangan Gagal

```
| ?- mainkanKartu(3).
Upin memainkan kartu: merah-skip

Pemain berikutnya kehilangan giliran.

--- Giliran Selesai ---
Giliran Apin

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. kuning-4
2. kuning-5
3. hijau-3
4. kuning-4
5. hitam-plus_empat
6. biru-8
7. hitam-wildcard

yes
| ?- mainkanKartu(5).
Apin memainkan kartu: hitam-plus_empat

Pilih warna:
1. Merah
2. Kuning
3. Hijau
4. Biru
Pilihan anda:
4.
Warna aktif sekarang adalah biru.
```

Kartu +4 dikeluarkan! Pemain berikutnya bisa menantang.

--- Giliran Selesai ---

Giliran Upin

(31 ms) yes

| ?- tantang.

Tantangan dilakukan!

Memeriksa kartu Apin...

Tantangan gagal!

Apin tidak memiliki kartu yang cocok.

Upin mendapatkan 6 kartu acak.

Upin mendapatkan kartu: merah-skip

Upin mendapatkan kartu: hijau-skip

Upin mendapatkan kartu: kuning-skip

Upin mendapatkan kartu: hitam-plus_empat

Upin mendapatkan kartu: biru-0

Upin mendapatkan kartu: merah-8

Giliran Ipin.

yes

Skenario 4 Tantangan Berhasil

| ?- cekInfo.

Kartu discard top: merah-9

Urutan pemain: Apin, Ipin, Upin

Informasi pemain:

- Apin: 12 kartu

- Ipin: 12 kartu

- Upin: 12 kartu

yes

| ?- lihatKartu.

Berikut kartu yang anda miliki.

1. kuning-7
2. biru-9
3. kuning-3
4. biru-2
5. hijau-6
6. biru-reverse
7. hijau-9
8. biru-8
9. kuning-plus_dua
10. merah-6
11. hijau-9
12. hitam-plus_empat

yes

| ?- mainkanKartu(12).

Apin memainkan kartu: hitam-plus_empat

Pilih warna:

1. Merah
2. Kuning
3. Hijau
4. Biru

Pilihan anda:

4.

Warna aktif sekarang adalah biru.

Kartu +4 dikeluarkan! Pemain berikutnya bisa menantang.

--- Giliran Selesai ---

Giliran Ipin

(31 ms) yes

| ?- tantang.

```
Tantangan dilakukan!
Memeriksa kartu Apin...
Tantangan berhasil!
Pemain Apin memiliki kartu yang cocok.
Pemain Apin harus mengambil 4 kartu.
Apin mendapatkan kartu: hitam-wildcard

Apin mendapatkan kartu: hitam-wildcard

Apin mendapatkan kartu: biru-8

Apin mendapatkan kartu: kuning-4

Giliran Upin.

yes
```

2.2.4 uni(NomorUrutKartuDiTangan).

Command uni(NomorUrut) . adalah *command* khusus yang diwajibkan bagi pemain untuk memainkan kartu kedua terakhir dari tangannya sekaligus menyerukan "UNI!". Kegagalan mematuhi syarat jumlah kartu saat mengeksekusi perintah ini akan langsung berbuah penalti. Alur logika *uni(NomorUrut)* . antara lain:

- Validasi Kondisi State Permainan:
Sistem mengecek apakah permainan sudah dimulai melalui basis fakta `game_started/0`.
- Validasi Syarat Jumlah Kartu (Penalti): Sistem akan menghitung panjang *list* kartu di tangan pemain. Perintah ini hanya valid jika jumlah kartu tepat berjumlah 2. Jika pemain mengeksekusi perintah ini dengan jumlah kartu selain 2, sistem akan menggagalkan pelemparan kartu, memberikan penalti berupa tambahan 1 kartu acak ke tangan, lalu secara paksa memindahkan giliran ke pemain berikutnya (misalnya, dari Apin langsung beralih ke Ipin).

- Validasi Kepemilikan dan Kecocokan Kartu:

Jika syarat jumlah kartu terpenuhi, sistem mengambil kartu berdasarkan nomor urut dan mencocokkannya dengan kartu di meja (`discard_top/1`). Sistem akan menolak jika warna atau jenis kartu tidak sesuai.

- Pencegahan Tumpuk Efek & Pembaruan Memori:

Menjalankan prosedur keamanan yang sama dengan `mainkanKartu`, yaitu menolak penumpukan `plus_dua` secara berturut-turut, serta mencatat pembaruan memori ke dalam fakta `warna_sebelumnya`, `jenis_sebelumnya`, dan `yg_keluarin_plus4` jika kartu yang dilempar adalah *Wild Draw Four* (+4).

- Pembaruan Status Keamanan UNI & Transisi State:

Jika semua validasi berhasil, kartu dibuang ke meja. Pemain yang berhasil mengeksekusi perintah ini akan didaftarkan ke dalam *dynamic predicate* `list_uni/1` sebagai tanda bahwa mereka aman dari incaran perintah tangkap pemain lain. Sistem lalu mengeksekusi efek kartu (jika ada) dan memindahkan giliran.

Skenario 1 (`uni`. dilakukan ketika kartu di tangan pemain tidak berjumlah 2):

Contoh Hasil Eksekusi Program
<pre> ?- startGame. Masukkan jumlah pemain: 3. Masukkan nama pemain 1: 'Apin'. Masukkan nama pemain 2: 'Ipin'. Masukkan nama pemain 3: 'Upin'. Ipin - Apin - Upin Setiap pemain mendapatkan 7 kartu acak Kartu discard top: merah-8</pre>

Giliran Ipin

(47 ms) yes

| ?- uni(1).

Syarat tidak terpenuhi! Kartu di tanganmu tidak berjumlah 2.

Ipin mendapatkan penalti 1 kartu acak dan kehilangan giliran

Giliran Apin

(16 ms) no

| ?-

Skenario 2 (uni. dilakukan saat jumlah kartu 2, tetapi kartu yang dipilih tidak cocok dengan meja):

Contoh Hasil Eksekusi Program

| ?- cekInfo.

Kartu discard top: biru-5

Urutan pemain: Ipin, Apin, Upin

Informasi pemain:

- Ipin: 2 kartu

- Apin: 7 kartu

- Upin: 7 kartu

(15 ms) yes

| ?- lihatKartu.

Berikut kartu yang anda miliki.

1. merah-9

2. kuning-3

yes

| ?- uni(1).

Kartu tidak valid! Warna atau angkanya tidak cocok dengan kartu di meja.

no

Skenario 3 (uni. berhasil dilakukan secara normal, pemain masuk daftar aman, dan sisa kartu menjadi 1):

Contoh Hasil Eksekusi Program

```
| ?- cekInfo.  
Kartu discard top: hijau-4  
Urutan pemain: Upin, Apin, Ipin  
Informasi pemain:  
- Upin: 2 kartu  
- Apin: 7 kartu  
- Ipin: 7 kartu  
  
yes  
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. hijau-4  
2. biru-1  
  
yes  
| ?- uni(1).  
Upin teriak "UNI!" dan memainkan kartu: hijau-4  
  
--- Giliran Selesai ---  
Giliran Apin  
  
yes
```

Skenario 4 (Uni tidak dapat dilakukan ketika terkena efek plus_empat atau plus_dua).

```

| ?- mainkanKartu(10).
Ipin memainkan kartu: hitam-plus_empat

Pilih warna:
1. Merah
2. Kuning
3. Hijau
4. Biru
Pilihan anda:
1.
Warna aktif sekarang adalah merah.
Kartu +4 dikeluarkan! Pemain berikutnya bisa menantang.

--- Giliran Selesai ---
Giliran Apin

(32 ms) yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-8
2. kuning-4

yes
| ?- uni(1).
Anda tidak dapat menyerukan UNI dan memainkan kartu saat terkena +4 atau
+2

no

```

2.2.5 tangkap>NamaPemain).

Command tangkap>NamaTarget). adalah perintah pendukung yang digunakan sebagai mekanisme penegakan aturan (hukuman) bagi pemain yang lalai. Perintah ini dimanfaatkan untuk menangkap pemain lain yang sudah memiliki tepat 1 sisa kartu di tangannya, namun lupa mendeklarasikan status keamanannya (lupa mengeksekusi perintah uni). Alur logika tangkap>NamaTarget). antara lain:

- Validasi Kondisi State Permainan:
Sistem memverifikasi apakah permainan sudah berjalan (fakta `game_started/0`).
- Pengecekan Syarat Jumlah Kartu Target:
Sistem menghitung panjang *list* `kartu_tangan` milik>NamaTarget).

Syarat mutlak pertama untuk penangkapan adalah jumlah kartu target harus tepat berjumlah 1.

- Pengecekan Status Keamanan (List UNI): Sistem mengecek apakah NamaTarget terdapat di dalam *dynamic predicate list_uni/1*. Jika pemain tersebut mengeksekusi perintah uni di giliran sebelumnya, namanya akan berada di list ini dan dinyatakan kebal (aman) dari penangkapan.
- Evaluasi Hukuman (Penalti Silang):
 - Jika tangkapan valid (kartu target sisa 1 dan namanya tidak ada di *list_uni*), target akan otomatis ditambahkan 2 kartu acak ke tangannya.
 - Jika tangkapan tidak valid (target memiliki lebih dari 1 kartu, atau target sudah aman di dalam *list_uni*), maka pemain yang menuduh (pemain yang mengeksekusi perintah tangkap) akan terkena hukuman *boomerang* berupa penambahan 1 kartu acak.
- Transisi Giliran (Memakan Giliran): Baik tangkapan tersebut berhasil maupun gagal, aksi ini akan langsung memakan giliran pemain yang mengeksekusinya, dan *turn* akan dilempar ke pemain selanjutnya dalam urutan.

Skenario 1 (*tangkap*. berhasil dilakukan: Target memiliki 1 kartu dan tidak ada di daftar aman):

Contoh Hasil Eksekusi Program
?- cekInfo. Kartu discard top: hijau-3 Urutan pemain: Ipin, Apin, Upin Informasi pemain: - Ipin: 7 kartu - Apin: 7 kartu

- Upin: 1 kartu

yes

| ?- lihatKartu.

Berikut kartu yang anda miliki.

1. merah-2

2. hijau-8

3. hijau-3

4. merah-7

5. merah-plus_dua

6. biru-2

7. biru-2

yes

| ?- tangkap('Upin').

Target tertangkap basah! Upin lupa bilang UNI!

Hukuman: Tambah 2 kartu ke tangan.

Giliran Apin

yes

Skenario 2 (tangkap. gagal dilakukan: Target aman karena sudah teriak UNI, atau sisa kartunya bukan 1):

Contoh Hasil Eksekusi Program

| ?- startGame.

Masukkan jumlah pemain: 3.

Masukkan nama pemain 1: 'Apin'.

Masukkan nama pemain 2: 'Ipin'.

Masukkan nama pemain 3: 'Upin'.

Ipin - Apin - Upin

Setiap pemain mendapatkan 7 kartu acak

Kartu discard top: biru-1

Giliran Ipin

yes

| ?- cekInfo.

Kartu discard top: biru-1

Urutan pemain: Ipin, Apin, Upin

Informasi pemain:

- Ipin: 7 kartu

- Apin: 7 kartu

- Upin: 7 kartu

(15 ms) yes

| ?- lihatKartu.

Berikut kartu yang anda miliki.

1. biru-6

2. kuning-6

3. hijau-2

4. biru-3

5. merah-1

6. merah-4

7. kuning-5

(31 ms) yes

| ?- tangkap('Ipin').

Gagal menangkap! Target aman atau kartunya tidak bersisa 1.

Hukuman: Ipin mendapatkan 1 kartu penalti karena salah tuduh!

Giliran Apin

yes

Skenario 3: tangkap. gagal karena target memiliki 1 kartu, tetapi dia sudah aman (sudah teriak UNI di gilirannya).

Contoh Hasil Eksekusi Program

```
| ?- cekInfo.  
Kartu discard top: kuning-1  
Urutan pemain: Apin, Ipin, Upin  
Informasi pemain:  
- Apin: 7 kartu  
- Ipin: 7 kartu  
- Upin: 1 kartu  
  
yes  
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. biru-6  
2. kuning-2  
3. biru-0  
4. hitam-wildcard  
5. hijau-0  
6. hitam-plus_empat  
7. merah-4  
  
yes  
| ?- tangkap('Upin').  
Gagal menangkap! Target aman atau kartunya tidak bersisa 1.  
Hukuman: Apin mendapatkan 1 kartu penalti karena salah tuduh!  
Giliran Ipin  
yes
```

2.3 Perintah Aksi Pendukung

Kelompok *command* pada bagian ini berfungsi sebagai *utility command* yang mendukung jalannya permainan tanpa secara langsung menjadi aksi utama turn pemain. *Command-command* ini digunakan untuk membantu menampilkan *state* permainan, mendeteksi pelanggaran aturan, menampilkan daftar *command* yang tersedia, serta menyediakan informasi *run-time* permainan kepada pemain selama sesi berlangsung.

2.3.1 lihatCommand.

Command “lihatCommand.” digunakan untuk melihat daftar yang dapat dijalankan pada suatu giliran aktif saat ini. Aksi-aksi tersebut terdiri dari dua kategori, yaitu aksi utama yang memicu pergantian giliran dan aksi pendukung yang selalu ada di setiap giliran. Aksi utama akan berubah menyesuaikan dengan kondisi kartu pada tangan pemain dan kartu teratas pada *discard pile*.

Secara umum, semua aksi utama yang tersedia dalam permainan adalah mainkanKartu(NomorUrut)., ambilKartu., tantang., tangkap(Pemain)., sembunyikanKartu(NomorUrut). uni(NomorUrut)., dan tampilkanKartu(NomorUrut).. Sementara itu, aksi pendukung yang tersedia adalah lihatCommand., lihatKartu., dan cekInfo..

Perubahan dinamis daftar aksi utama pada lihatCommand. adalah sebagai berikut:

- Kondisi pemain memiliki kartu yang cocok dengan kartu teratas *discard pile* dan tidak terkena kartu aksi apapun sehingga pemain bisa memilih semua aksi yang dapat dilakukan.

Skenario 1 pemain memiliki kartu yang cocok

Contoh Hasil Eksekusi Program
<pre> ?- cekInfo. Kartu discard top: biru-8 Urutan pemain: Ipin, Apin, Upin Informasi pemain: - Ipin: 8 kartu - Apin: 7 kartu - Upin: 7 kartu yes ?- lihatKartu. Berikut kartu yang anda miliki.</pre>


```

1. merah-7
2. hitam-plus_empat
3. hitam-plus_empat
4. merah-0
5. biru-2
6. biru-1
7. hitam-plus_empat

yes
| ?- lihatCommand.
Aksi utama yang tersedia:
1. mainkanKartu(NomorUrut)
2. ambilKartu
3. tangkap(Pemain)
4. sembunyikanKartu(NomorUrut)

Aksi pendukung yang tersedia:
1. lihatCommand
2. lihatKartu
3. cekInfo

yes

```

- Kondisi kartu teratas *discard pile* adalah plus 2 sehingga pemain hanya dapat memberikan *command ambilKartu..*

Skenario 2 pemain terkena efek plus 2

Contoh Hasil Eksekusi Program
<pre> ?- mainkanKartu(8). Apin memainkan kartu: kuning-plus_dua --- Giliran Selesai --- Giliran Upin </pre>

```
yes
| ?- lihatCommand.
Aksi utama yang tersedia:
1. ambilKartu

Aksi pendukung yang tersedia:
1. lihatCommand
2. lihatKartu
3. cekInfo

(31 ms) yes
```

- Kondisi kartu teratas *discard pile* adalah plus 4 sehingga pemain hanya punya 2 pilihan yaitu **ambilKartu**. dan **tantang..**

Skenario 3 pemain terkena efek plus 4

Contoh Hasil Eksekusi Program
<pre> ?- mainkanKartu(4). Upin memainkan kartu: hitam-plus_empat Pilih warna: 1. Merah 2. Kuning 3. Hijau 4. Biru Pilihan anda: 3. Warna aktif sekarang adalah hijau. Kartu +4 dikeluarkan! Pemain berikutnya bisa menantang. --- Giliran Selesai --- Giliran Ipin (31 ms) yes</pre>

```
| ?- lihatCommand.
Aksi utama yang tersedia:
1. ambilKartu
2. tantang

Aksi pendukung yang tersedia:
1. lihatCommand
2. lihatKartu
3. cekInfo

(16 ms) yes
```

- Kondisi ketika tidak ada kartu di tangan yang cocok dengan *discard pile* Sehingga pemain mempunyai pilihan untuk `ambilKartu()`, `tangkap()`, serta opsi `sembunyikanKartu()` dan `tampilkanKartu()` jika memenuhi syarat masing-masing.

Skenario 4 pemain tidak memiliki kartu yang cocok

Contoh Hasil Eksekusi Program

```
| ?- cekInfo.
Kartu discard top: hitam-wildcard
Warna aktif: biru
Urutan pemain: Upin, Ipin, Apin
Informasi pemain:
- Upin: 7 kartu
- Ipin: 12 kartu
- Apin: 6 kartu

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-7
2. hijau-8
3. hijau-6
```

```

4. hijau-6
5. hitam-wildcard
6. hijau-4

(15 ms) yes
| ?- lihatCommand.
Aksi utama yang tersedia:
1. ambilKartu
2. tangkap(Pemain)
3. sembunyikanKartu(NomorUrut)

Aksi pendukung yang tersedia:
1. lihatCommand
2. lihatKartu
3. cekInfo

(31 ms) yes

```

- Kondisi untuk `sembunyikanKartu`, `sembunyikanKartu` akan ditampilkan apabila jumlah kartu di tangan pemain lebih dari 1 kartu.

Skenario 5 `sembunyikanKartu` ditampilkan

Contoh Hasil Eksekusi Program
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. merah-7 2. hijau-8 3. kuning-2 4. hijau-6 5. hijau-6 6. hitam-wildcard 7. hijau-4 yes ?- lihatCommand. </pre>

```
Aksi utama yang tersedia:
1. mainkanKartu(NomorUrut)
2. ambilKartu
3. tangkap(Pemain)
4. sembunyikanKartu(NomorUrut)

Aksi pendukung yang tersedia:
1. lihatCommand
2. lihatKartu
3. cekInfo

yes
```

- Kondisi untuk `tampilkanKartu`, `tampilkanKartu` akan ditampilkan apabila ada setidaknya 1 kartu yang disembunyikan.

Skenario 6 `tampilkanKartu` ditampilkan

Contoh Hasil Eksekusi Program
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. merah-7 2. hijau-8 3. hijau-6 4. hijau-6 5. hitam-wildcard (disembunyikan) 6. hijau-4 yes ?- lihatCommand. Aksi utama yang tersedia: 1. mainkanKartu(NomorUrut) 2. ambilKartu 3. tangkap(Pemain) 4. sembunyikanKartu(NomorUrut) 5. tampilkanKartu(NomorUrut)</pre>

Aksi pendukung yang tersedia:

1. lihatCommand
2. lihatKartu
3. cekInfo

yes

Skenario 7 pemain memiliki 2 Kartu dan Kartu yang dimiliki valid untuk dimainkan

Ipin memainkan kartu: merah-8

--- Giliran Selesai ---

Giliran Apin

yes

| ?- lihatCommand.

Aksi utama yang tersedia:

1. mainkanKartu(NomorUrut)
2. ambilKartu
3. tangkap(Pemain)
4. uni(NomorUrut)
5. sembunyikanKartu(NomorUrut)

Aksi pendukung yang tersedia:

1. lihatCommand
2. lihatKartu
3. cekInfo

yes

| ?- lihatKartu.

Berikut kartu yang anda miliki.

1. merah-8
2. kuning-4

yes

2.3.2 lihatKartu.

Command `lihatKartu.` digunakan untuk melihat daftar dan status seluruh kartu yang sedang berada di tangan pemain pada suatu giliran aktif saat ini. Melalui *command* ini, pemain dapat melihat semua isi kartu yang mereka punya beserta nomor

urut yang nantinya digunakan sebagai parameter *command* `mainkanKartu(NomorUrut)`.

Mekanisme penampilan daftar kartu diatur berdasarkan aturan yang spesifik. Setiap kartu yang ada di tangan pemain akan ditampilkan secara berurutan mulai dari nomor 1 hingga jumlah maksimal kartu yang dipegang. Nomor urut ini akan secara otomatis bergeser atau bertambah sesuai dengan kondisi permainan kartu pemain. Selain menampilkan warna dan jenis kartu, `lihatKartu` juga memperlihatkan apabila ada kartu yang disembunyikan pada pemain dengan menampilkan keterangan “(disembunyikan)” pada samping kartu.

Skenario 1 tidak ada kartu yang disembunyikan

Contoh Hasil Eksekusi Program
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. hijau-reverse 2. merah-1 3. kuning-reverse 4. hijau-plus_dua 5. biru-skip 6. hijau-5 7. hitam-plus_empat (15 ms) yes</pre>

Skenario 2 ada kartu yang disembunyikan

Contoh Hasil Eksekusi Program
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. merah-4 2. kuning-6 (disembunyikan) 3. kuning-3</pre>

```
4. biru-3
5. hijau-6 (disembunyikan)
6. biru-1
7. hitam-wildcard
8. hijau-reverse
9. biru-skip
10. merah-reverse (disembunyikan)
11. merah-2

(16 ms) yes
```

2.3.3 cekInfo.

Command cekInfo. digunakan untuk menampilkan informasi permainan saat ini secara real-time berdasarkan state yang tersimpan di dalam *run-time memory*. Informasi yang ditampilkan meliputi kartu teratas pada *discard pile*, urutan giliran pemain, serta jumlah kartu yang dimiliki masing-masing pemain.

- Informasi kartu teratas *discard pile*

Predikat `discard_top/1` digunakan untuk mengambil objek *compound term* kartu teratas dari *discard pile* dalam bentuk `kartu(WarnaTeratas, JenisTeratas, _)`. Nilai warna dan jenis kartu kemudian diekstraksi melalui proses *pattern matching* dan ditampilkan menggunakan operasi output `write/1`. Jika sistem mendeteksi bahwa `WarnaTeratas == 'hitam'`, yang berarti adalah *wild card*, sistem secara otomatis mengevaluasi kondisi percabangan *if-else* (`-> ;`) untuk mengambil fakta `warna_wild(WarnaAktif)` yang sedang dipilih oleh pemain, lalu menampilkan warna aktif tersebut di bawah info kartu teratas.

- Informasi urutan pemain

Urutan giliran pemain diperoleh dari basis fakta `urutan_pemain/1`, lalu ditampilkan menggunakan predikat rekursif `print_list_pemain/1`. Predikat ini melakukan traversal list pemain dari kepala hingga elemen terakhir.

Selama proses traversal berlangsung, setiap nama pemain dicetak secara berurutan dan dipisahkan menggunakan delimiter koma.

- Informasi jumlah kartu pemain lain

Informasi jumlah kartu setiap pemain ditampilkan melalui predikat rekursif `print_info_pemain/1`. Sistem melakukan traversal terhadap seluruh daftar pemain, kemudian mengambil daftar kartu masing-masing pemain melalui fakta `kartu_tangan>NamaPemain, ListKartuTangan`). Jumlah kartu dihitung menggunakan predikat `panjang/3`, lalu ditampilkan dalam format “NamaPemain: N kartu”.

Skenario 1 (Kartu teratas adalah kartu warna biasa)

```
| ?- cekInfo.  
Kartu discard top: merah-7  
Urutan pemain: Upin, Apin, Ipin  
Informasi pemain:  
- Upin: 7 kartu  
- Apin: 7 kartu  
- Ipin: 7 kartu  
  
yes
```

Skenario 2 (Kartu teratas adalah kartu wild)

```
| ?- cekInfo.  
Kartu discard top: hitam-wildcard  
Warna aktif: hijau  
Urutan pemain: Upin, Apin, Ipin  
Informasi pemain:  
- Upin: 6 kartu  
- Apin: 7 kartu  
- Ipin: 5 kartu  
  
yes
```

2.4 Fitur Bonus

2.4.1 sembunyikanKartu(NomorUrut).

Command `sembunyikanKartu(NomorUrut)` digunakan oleh pemain untuk menyembunyikan kartu tertentu yang ada di tangan mereka dari penglihatan pemain lain. *Command* ini termasuk ke dalam kategori aksi utama yang memicu pergantian giliran setelah berhasil dieksekusi. *Command* ini tidak dapat dijalankan ketika pemain hanya memiliki 1 kartu di tangan. Keterangan “(disembunyikan)” akan terlihat pada kartu yang disembunyikan ketika pemain menjalankan *command* `lihatKartu`. Pemain tidak dapat menyembunyikan kartu yang telah disembunyikan sebelumnya.

- Skenario 1 menyembunyikan kartu di tengah turn (tanpa efek `plus_dua` atau `plus_empat`)

Contoh Hasil Eksekusi Program
<pre> ?- sembunyikanKartu(1). Kartu kuning-6 berhasil disembunyikan. Giliran Apin yes ?- ambilKartu. Apin mendapatkan kartu: kuning-5 Giliran Ipin yes ?- ambilKartu. Ipin mendapatkan kartu: merah-2 Giliran Upin yes ?- lihatKartu. Berikut kartu yang anda miliki. 1. kuning-6 (disembunyikan)</pre>

```
2. merah-1
3. kuning-3
4. merah-4
5. biru-5
6. hijau-5
7. hijau-4
8. merah-2
9. merah-reverse
10. kuning-9
11. kuning-7

yes
```

- Skenario 2 menyembunyikan kartu saat terkena efek **plus_empat**

Contoh Hasil Eksekusi Program

```
| ?- mainkanKartu(5).
Apin memainkan kartu: hitam-plus_empat

Pilih warna:
1. Merah
2. Kuning
3. Hijau
4. Biru
Pilihan anda:
3.
Warna aktif sekarang adalah hijau.
Kartu +4 dikeluarkan! Pemain berikutnya bisa menantang.

--- Giliran Selesai ---
Giliran Ipin

(31 ms) yes
| ?- sembunyikanKartu(1).
Anda tidak dapat menyembunyikan kartu saat ini!
```

no

- Skenario 3 menyembunyikan kartu saat terkena efek `plus_dua`

Contoh Hasil Eksekusi Program

```
| ?- mainkanKartu(4).  
Apin memainkan kartu: kuning-plus_dua  
  
--- Giliran Selesai ---  
Giliran Ipin  
  
yes  
| ?- sembunyikanKartu(1).  
Anda tidak dapat menyembunyikan kartu saat ini!  
  
no
```

- Skenario 4 menyembunyikan kartu yang telah disembunyikan

```
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. merah-8 (disembunyikan)  
2. hijau-8  
3. hitam-wildcard  
4. hitam-plus_empat  
5. kuning-5  
6. merah-plus_dua  
7. kuning-2  
8. hijau-2  
  
yes  
| ?- sembunyikanKartu(1).  
Kartu tersebut memang sudah tersembunyi!  
  
no
```

2.4.2 tampilkanKartu(NomorUrut).

Command tampilkanKartu(NomorUrut) digunakan untuk menampilkan kembali kartu yang sebelumnya telah disembunyikan. *Command* ini juga termasuk ke dalam kategori aksi utama yang memicu pergantian giliran setelah berhasil dieksekusi. Pemain tidak dapat menampilkan kartu yang sebelumnya tidak disembunyikan.

- Skenario 1 (Kartu tidak dalam kondisi hide) :

Contoh Hasil Eksekusi Program
?- tampilkanKartu(1). Kartu tersebut memang sudah ditampilkan! no

- Skenario 2 (Kartu dalam kondisi hide)

Contoh Hasil Eksekusi Program
?- tampilkanKartu(1). Kartu hitam-wildcard berhasil ditampilkan! Giliran Upin yes

- Skenario 3 (tampilkan kartu saat terkena plus_empat atau plus_dua)

Contoh Hasil Eksekusi Program
?- mainkanKartu(6). Upin memainkan kartu: hitam-plus_empat Pilih warna:

1. Merah
2. Kuning
3. Hijau
4. Biru

Pilihan anda:

3.

Warna aktif sekarang adalah hijau.

Kartu +4 dikeluarkan! Pemain berikutnya bisa menantang.

--- Giliran Selesai ---

Giliran Ipin

(31 ms) yes

| ?- tampilkanKartu(1).

Anda tidak dapat menampilkan kartu saat ini!

no

| ?- mainkanKartu(3).

Apin memainkan kartu: kuning-plus_dua

--- Giliran Selesai ---

Giliran Ipin

(16 ms) yes

| ?- tampilkanKartu(1).

Anda tidak dapat menampilkan kartu saat ini!

no

Bab III

Hasil Eksekusi Program

3.1 Inisialisasi Permainan

Fase inisiasi permainan dilakukan dengan mengeksekusi *command* `startGame..` Pada fase ini, sistem akan mereset seluruh *state* permainan sebelumnya (memanfaatkan `retractall` pada fakta dinamis) dan memulai sesi permainan yang sepenuhnya baru.

Program akan meminta masukan jumlah pemain dengan batasan validasi antara 2 hingga 4 orang. Selanjutnya, sistem meminta masukan nama untuk setiap pemain. Validasi ketat diterapkan di sini: nama pemain harus berwujud *atomic* dan wajib unik. Jika terdapat nama yang terduplikasi, sistem akan menolak dan meminta input ulang.

Setelah seluruh data pemain tervalidasi, sistem akan mengacak urutan giliran bermain, membagikan tepat 7 kartu secara acak ke dalam `kartu_tangan` masing-masing pemain, dan menarik satu kartu sebagai kartu awal di meja (*discard top*). Kartu awal tersebut sudah dipastikan bukan merupakan kartu aksi (*skip*, *reverse*, *plus dua*, *wildcard*, *plus empat*). Fase ini diakhiri dengan deklarasi pemain pertama yang mendapatkan hak untuk memulai giliran.

Contoh Tampilan Inisialisasi Permainan

```
| ?- startGame.  
Masukkan jumlah pemain: 3.  
  
Masukkan nama pemain 1: 'Apin'.  
Masukkan nama pemain 2: 'Ipin'.  
Masukkan nama pemain 3: 'Apin'.  
Nama sudah digunakan. Masukan nama lain: 'Upin'.  
Upin - Apin - Ipin
```

```
Setiap pemain mendapatkan 7 kartu acak
```

```
Kartu discard top: merah-0
```

```
Giliran Upin
```

```
(31 ms) yes
```

3.2 Siklus Turn Permainan

Siklus *turn* permainan merupakan inti dari berjalannya program, di mana hak untuk melakukan aksi berotasi antar pemain berdasarkan urutan dan arah permainan yang sedang aktif (bisa searah atau berlawanan akibat efek kartu *Reverse*). Pada saat gilirannya, seorang pemain diberikan kebebasan untuk mengeksekusi berbagai aksi pendukung (seperti `lihatKartu.`, `cekInfo.`, atau `lihatCommand.`) secara berkali-kali tanpa memakan gilirannya.

Siklus giliran seorang pemain hanya akan dinyatakan selesai (*turn ended*) apabila ia berhasil mengeksekusi satu aksi utama yang tervalidasi. Aksi utama tersebut meliputi pembuangan kartu (`mainkanKartu().` atau `uni().`), pengambilan kartu dari *deck* (`ambilKartu.`), pengecekan kartu +4 (`tantang.`), menangkap pemain yang lupa menyerukan *uni* (`tangkap.`), menyembunyikan kartu (`sembunyikanKartu().`), atau menampilkan kembali kartu yang sebelumnya disembunyikan (`tampilkanKartu().`).

Setelah salah satu aksi utama ini dieksekusi dan dievaluasi oleh sistem, program akan secara otomatis mengaktifkan efek kartu (jika ada), memperbarui *state* permainan di memori, mencetak pesan "*--- Giliran Selesai ---*", dan mendeklarasikan nama pemain berikutnya yang mendapatkan hak aksi.

Contoh Tampilan Siklus Turn Permainan
<pre> ?- cekInfo. Kartu discard top: biru-2 Urutan pemain: Upin, Apin, Ipin Informasi pemain: - Upin: 2 kartu - Apin: 7 kartu - Ipin: 7 kartu yes ?- lihatKartu. Berikut kartu yang anda miliki. 1. biru-5 2. kuning-1 yes ?- mainkanKartu(1). Upin memainkan kartu: biru-5 --- Giliran Selesai --- Giliran Apin yes </pre>

3.3 Fitur Khusus

Bagian ini memuat implementasi mekanik permainan yang menjadi nilai tambah atau modifikasi khusus dari aturan standar, yang dirancang untuk menambah kompleksitas dan kedalaman strategi. Fitur khusus dalam program ini meliputi:

3.3.1 Mekanisme Sembunyikan dan Tampilkan Kartu

Sistem menyediakan utilitas taktis melalui *command* `sembunyikanKartu(NomorUrut)`. dan `tampilkanKartu(NomorUrut)`.. Pemain dapat menyembunyikan status kartu dari tampilan standar untuk alasan strategi. Kartu yang berstatus "disembunyikan" tetap dapat dimainkan ke meja (*discard pile*) meskipun belum dikembalikan ke status normal menggunakan perintah `tampilkanKartu()`.. Fitur ini berguna untuk mengelabui lawan ketika mereka memeriksa jumlah kartu pemain dengan perintah `cekInfo`..

Contoh Tampilan Mekanisme Sembunyikan dan Tampilkan Kartu

```
| ?- startGame.
Masukkan jumlah pemain: 3.

Masukkan nama pemain 1: 'Apin'.
Masukkan nama pemain 2: 'Ipin'.
Masukkan nama pemain 3: 'Upin'.
Ipin - Apin - Upin

Setiap pemain mendapatkan 7 kartu acak

Kartu discard top: hijau-6

Giliran Ipin

(16 ms) yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. hijau-reverse
2. kuning-4
3. hitam-plus_empat
4. biru-1
5. merah-plus_dua
```

```

6. hitam-wildcard
7. merah-5

(16 ms) yes
| ?- sembunyikanKartu(1).
Kartu hijau-reverse berhasil disembunyikan.

Giliran Apin

yes

```

3.3.2 Eksekusi *Wildcard* dan *Plus Four* (+4)

Sistem mengimplementasikan eksekusi interaktif untuk kartu *Wildcard* dan *Draw Four* (+4). Ketika kartu ini dimainkan, program akan menahan perpindahan giliran. Pemain diwajibkan menginput nomor yang merepresentasikan warna (1. Merah, 2. Kuning, 3. Hijau, 4. Biru). Input ini akan mengubah fakta memori `warna_aktif`, yang nantinya akan menjadi patokan validasi kartu bagi pemain yang mendapat giliran selanjutnya. Jika Pemain menggunakan *Draw Four*, selain pemain dapat memilih warna, ia juga memberikan efek('plus_empat'). terhadap pemain berikutnya sehingga pemain berikutnya dapat menantang maupun mengambil kartu sebanyak 4.

Contoh Tampilan Eksekusi Wildcard

```

| ?- mainkanKartu(1).
Upin memainkan kartu: hitam-wildcard

Pilih warna:
1. Merah
2. Kuning

```

3. Hijau
4. Biru
Pilihan anda:
1.
Warna aktif sekarang adalah merah.

--- Giliran Selesai ---
Giliran Apin

yes
| ?- cekInfo.
Kartu discard top: hitam-wildcard
Warna aktif: merah
Urutan pemain: Upin, Apin, Ipin
Informasi pemain:
- Upin: 1 kartu
- Apin: 7 kartu
- Ipin: 7 kartu

yes

Contoh Tampilan Eksekusi Plus Four (+4)

| ?- cekInfo.
Kartu discard top: kuning-8
Urutan pemain: Apin, Ipin, Upin
Informasi pemain:
- Apin: 2 kartu
- Ipin: 7 kartu
- Upin: 7 kartu

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.

```
1. hitam-plus_empat
2. kuning-6

(15 ms) yes
| ?- mainkanKartu(1).
Apin memainkan kartu: hitam-plus_empat

Pilih warna:
1. Merah
2. Kuning
3. Hijau
4. Biru
Pilihan anda:
3.
Warna aktif sekarang adalah hijau.
Kartu +4 dikeluarkan! Pemain berikutnya bisa menantang.

--- Giliran Selesai ---
Giliran Ipin

yes
```

3.3.3 Pencegahan Penumpukan Hukuman dan Kartu *Wild*

Sebagai bentuk penyeimbang permainan, program memiliki aturan khusus yang menolak penumpukan kartu spesial yang sejenis. Program akan mencegah pelemparan kartu *Plus Two* (+2) di atas kartu +2, *Wild Draw Four* (+4) di atas kartu +4, maupun *Wildcard* di atas *Wildcard* lainnya. Program mengevaluasi *history* jenis kartu terakhir di meja, dan akan memblokir aksi tersebut untuk mencegah pemain selanjutnya terkena akumulasi penalti yang merusak ritme permainan.

Contoh Tampilan Penolakan Kartu *Plus Two* (+2)

```

| ?- cekInfo.
Kartu discard top: kuning-plus_dua
Urutan pemain: Apin, Upin, Ipin
Informasi pemain:
- Apin: 2 kartu
- Upin: 7 kartu
- Ipin: 7 kartu

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. hijau-plus_dua
2. merah-5

yes
| ?- mainkanKartu(1).
Kartu +2 tidak boleh dimainkan berturut-turut!

no

```

Contoh Tampilan Penolakan Kartu Wild Draw Four (+4)

```

| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. merah-8
2. hijau-8
3. hitam-wildcard
4. hitam-plus_empat
5. hitam-wildcard
6. kuning-5
7. merah-plus_dua
8. kuning-2

```

```
yes
| ?- cekInfo.
Kartu discard top: hitam-plus_empat
Warna aktif: kuning
Urutan pemain: Upin, Ipin, Apin
Informasi pemain:
- Upin: 12 kartu
- Ipin: 8 kartu
- Apin: 6 kartu

yes
| ?- mainkanKartu(4).
Kartu +4 tidak boleh dimainkan berturut-turut!

no
```

Contoh Tampilan Penolakan Kartu *Wildcard*

```
| ?- cekInfo.
Kartu discard top: hitam-wildcard
Warna aktif: kuning
Urutan pemain: Ipin, Apin, Upin
Informasi pemain:
- Ipin: 4 kartu
- Apin: 6 kartu
- Upin: 7 kartu

yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. hitam-wildcard (disembunyikan)
2. kuning-reverse
3. hijau-reverse
4. hitam-wildcard
5. biru-5
```

```

6. hitam-plus_empat
7. merah-plus_dua
8. hitam-plus_empat

(15 ms) yes
| ?- mainkanKartu(1).
Kartu wildcard tidak boleh dimainkan berturut-turut!

no

```

3.3.4 Eksekusi Efek Kartu Reverse dan Skip

Pengujian ini dilakukan untuk melakukan proses skip maupun reverse di dalam eksekusi program. Efek Aksi Reverse berguna untuk mengubah urutan jalannya permainan, misal dari kanan menjadi ke kiri, misal ($A \rightarrow B \rightarrow C \rightarrow A$ menjadi $C \rightarrow B \rightarrow A \rightarrow C$). Namun, aksi reverse dapat dianggap juga sebagai aksi skip jika hanya dimainkan oleh 2 orang saja. Di sisi lain, aksi skip digunakan untuk menghilangkan giliran pemain setelah pemain saat ini.

Eksekusi Efek Aksi Reverse dalam 3 Player

```

| ?- startGame.
Masukkan jumlah pemain: 3.

Masukkan nama pemain 1: 'Apin'.
Masukkan nama pemain 2: 'Ipin'.
Masukkan nama pemain 3: 'Upin'.
Upin - Ipin - Apin

Setiap pemain mendapatkan 7 kartu acak

Kartu discard top: biru-0

Giliran Upin

(16 ms) yes
| ?- lihatKartu.
Berikut kartu yang anda miliki.
1. biru-reverse
2. hitam-plus_empat
3. hijau-5
4. hitam-plus_empat
5. hitam-plus_empat

```



```
6. biru-6
7. biru-reverse

(16 ms) yes
| ?- cekInfo.
Kartu discard top: biru-0
Urutan pemain: Upin, Ipin, Apin
Informasi pemain:
- Upin: 7 kartu
- Ipin: 7 kartu
- Apin: 7 kartu

(15 ms) yes
| ?- mainkanKartu(1).
Upin memainkan kartu: biru-reverse

--- Giliran Selesai ---
Giliran Apin

(15 ms) yes
| ?- cekInfo.
Kartu discard top: biru-reverse
Urutan pemain: Apin, Ipin, Upin
Informasi pemain:
- Upin: 6 kartu
- Ipin: 7 kartu
- Apin: 7 kartu

(16 ms) yes
```

Efek Aksi Reverse Pada Kasus 2 Player (Berlaku seperti skip)

```
| ?- cekInfo.
Kartu discard top: hitam-plus_empat
Warna aktif: kuning
Urutan pemain: B, A
Informasi pemain:
- B: 9 kartu
- A: 6 kartu

yes
| ?- mainkanKartu(5).
B memainkan kartu: kuning-reverse

Pemain berikutnya kehilangan giliran.

--- Giliran Selesai ---
Giliran B

yes
```

Efek Aksi Skip
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. merah-plus_dua 2. kuning-9 3. hijau-skip 4. hijau-7 5. hijau-plus_dua 6. hitam-wildcard yes ?- mainkanKartu(3). Apin memainkan kartu: hijau-skip Pemain berikutnya kehilangan giliran. --- Giliran Selesai --- Giliran Upin yes ?- cekInfo. Kartu discard top: hijau-skip Urutan pemain: Apin, Ipin, Upin Informasi pemain: - Upin: 7 kartu - Ipin: 8 kartu - Apin: 5 kartu (16 ms) yes </pre>

3.3.5 Kombinasi Perintah tangkap., uni., dan tantang.

Pengujian ini dilakukan untuk melihat jalannya program ketika beberapa perintah khusus dieksekusi secara berurutan dalam satu putaran permainan.

Contoh Tampilan Kombinasi Perintah tangkap., uni., dan tantang.
<pre> ?- uni(2). Apin teriak "UNI!" dan memainkan kartu: hitam-plus_empat Pilih warna: 1. Merah 2. Kuning 3. Hijau 4. Biru </pre>

Pilihan anda:

1.

Warna aktif sekarang adalah merah.

Kartu +4 dikeluarkan! Pemain berikutnya bisa menantang.

--- Giliran Selesai ---

Giliran Apin

yes

| ?- tantang.

Tantangan dilakukan!

Memeriksa kartu Upin...

Tantangan berhasil!

Pemain Upin memiliki kartu yang cocok.

Pemain Upin harus mengambil 4 kartu.

Upin mendapatkan kartu: hitam-plus_empat

Upin mendapatkan kartu: kuning-5

Upin mendapatkan kartu: kuning-5

Upin mendapatkan kartu: hitam-wildcard

Giliran Ipin.

(16 ms) yes

| ?- cekInfo.

Kartu discard top: hitam-plus_empat

Warna aktif: merah

Urutan pemain: Upin, Apin, Ipin

Informasi pemain:

- Upin: 11 kartu

- Apin: 1 kartu

- Ipin: 7 kartu

yes

| ?- lihatKartu.

```
Berikut kartu yang anda miliki.
```

1. hijau-0
2. kuning-8
3. merah-2
4. hitam-plus_empat
5. hijau-6
6. merah-reverse
7. kuning-plus_dua

```
yes
```

```
| ?- tangkap('Apin').
```

```
Gagal menangkap! Target aman atau kartunya tidak bersisa 1.
```

```
Hukuman: Ipin mendapatkan 1 kartu penalti karena salah tuduh!
```

```
Giliran Upin
```

```
yes
```

Pada hasil eksekusi program di atas ketiga fitur ini diuji dalam satu rangkaian kejadian:

- Penggunaan uni: Pemain (Apin) membuang kartu *Wild Draw Four* (+4) menggunakan aksi utama `uni(2)`.. Sistem menerima kartu tersebut, mendaftarkan Apin ke dalam daftar aman sementara, dan memunculkan status peringatan kartu +4.
- Penggunaanantang: Peringatan kartu +4 tersebut direspons dengan perintah `antang`.. Sistem mengevaluasi status permainan dan mendeteksi adanya kartu yang cocok pada pemain sebelumnya (Upin), sehingga membatalkan efek awal dan menjatuhkan hukuman penalti penarikan 4 kartu kepada Upin. Giliran kemudian berlanjut ke Ipin.
- Penggunaan tangkap (Skenario Salah Tuduh): Pada gilirannya, Ipin mencoba menjalankan perintah `tangkap('Apin')`. karena Apin menyisakan 1 kartu. Namun, karena Apin telah mendeklarasikan status amannya melalui perintah `uni()`. sebelumnya, sistem menggagalkan penangkapan tersebut. Sebagai kompensasi atas tuduhan yang salah, sistem memberikan penalti kartu kepada Ipin.

3.4 Save and Load Game

Fitur (*Command*) ini digunakan untuk menyimpan progress pertandingan saat ini dengan `saveGame..` Kemudian, pemain dapat memainkan sesi ini kembali saat dilakukan `loadGame..` sesuai dengan nama *file* yang telah ditentukan sebelumnya.

Contoh Tampilan Save and Load Game

```
| ?- cekInfo.  
Kartu discard top: kuning-reverse  
Urutan pemain: Upin, Apin, Ipin  
Informasi pemain:  
- Ipin: 8 kartu  
- Apin: 6 kartu  
- Upin: 7 kartu  
  
(31 ms) yes  
| ?- saveGame.  
Masukkan nama file penyimpanan: sesi_seru.  
Status permainan berhasil disimpan ke sesi_seru.txt.  
  
yes  
| ?- loadGame.  
Masukkan nama file yang akan dimuat: sesi_seru.  
Status permainan berhasil dimuat dari sesi_seru.txt.  
Melanjutkan Giliran Ipin.  
  
(16 ms) yes  
| ?- cekInfo.  
Kartu discard top: kuning-reverse  
Urutan pemain: Upin, Apin, Ipin  
Informasi pemain:  
- Ipin: 8 kartu  
- Apin: 6 kartu  
- Upin: 7 kartu  
  
yes
```

Isi sesi_seru.txt
<pre> urutan_pemain: ['Ipin', 'Apin', 'Upin']. giliran: 'Ipin'. discard_top: kuning-reverse. warna_aktif: kuning. arah_permainan: kiri. status_UNI: []. kartu('Ipin'): [biru-5, hijau-5, hitam-wildcard, hijau-1, hijau-9, biru-5, biru-2, hitam-wildcard]. kartu('Apin'): [kuning-plus_dua, merah-0, kuning-6, hijau-4, hijau-7, kuning-0]. kartu('Upin'): [biru-1, hijau-5, hijau-3, hijau-4, biru-7, biru-5, kuning-5]. </pre>

3.5 Akhir Permainan

Fase akhir permainan terjadi ketika salah satu pemain berhasil menghabiskan seluruh kartu di tangan. Pada kondisi ini, *command* `endGame`. dijalankan untuk menghentikan permainan sekaligus melakukan kalkulasi skor akhir seluruh pemain.

Sistem menghitung total poin sisa kartu menggunakan `total_poin_pemain/2` dan `hitung_list_poin/2`. Seluruh kartu pemain dievaluasi satu per satu secara rekursif untuk menghasilkan total skor akhir. Setelah seluruh poin dihitung, sistem melakukan pengurutan ranking menggunakan `sort_pemain/2`.

Contoh Tampilan Akhir
<pre> Permainan selesai! Apin menghabiskan semua kartunya! Berikut perhitungan poin sisa kartu. Upin: merah-7 + hijau-skip = 7 + 10 = 17 poin Ipin: kuning-2 + hitam-wildcard = 2 + 20 = 22 poin Apin: kartu habis = 0 poin Urutan pemenang: 1. Apin (0 poin) 2. Upin (17 poin) </pre>

3. Ipin (22 poin)

Selamat, Apin menjadi pemenang!

yes

Setelah fase ini selesai, status permainan dinonaktifkan dan *run-time* kembali ke kondisi siap untuk menjalankan permainan baru.

Lampiran

- Repository Github

[IF1221_G25_ZeriUniDisTriz](#)

- Pembagian kerja

No.	Nama	NIM	Tugas yang dikerjakan
1.	Theresia Estelina Ratu Udju	13525088	Pada Bagian Program: - Pembuatan Rule dan Fakta Lihat Kartu - Pembuatan Rule dan Fakta Lihat Command - Pembuatan Rule dan Fakta Action Card plus_dua, skip, dan reverse - Pembuatan Rule dan Fakta Sembunyikan - Melakukan Perbaikan Bug dan Testing Edge Case Pada Bagian Laporan: - BAB II 2.3.1: lihatCommand. dan 2.3.2 lihatKartu. - BAB II: (2.4.1) sembunyikanKartu. dan (2.4.2) tampilkanKartu.
2.	Edward Terrance Lie	13525127	Pada Bagian Program: - Pembagian Tugas Fitur

			- Pembuatan Rule dan Fakta StartGame - Pembuatan Rule dan Fakta Ambil Kartu - Pembuatan Rule dan Fakta Save and Load Game - Mereview Code dan menginstruksikan perbaikan kepada kelompok - Melakukan Perbaikan Bug dan Testing Edge Case Pada Bagian Laporan: - BAB II 2.2.2 - 2.2.3: ambilKartu. dan tantang. - BAB 3.4 : Save dan Load Game - BAB V: Lampiran Form Penggunaan AI
3.	Christabelcyne Costan	13525141	Pada Bagian Program: - Pembuatan Rule dan Fakta CekInfo - Pembuatan Rule dan Fakta EndGame - Pembuatan Rule dan Fakta Action Card Wildcard, wild

			plus_empat. - Pembuatan Rule dan Fakta Tantang Pada Bagian Laporan: - BAB II : (2.1.1) startGame, (2.1.2) saveGame, (2.1.3) loadGame, dan (2.1.4) endGame. - BAB II (2.3.3) : cekInfo. - BAB 3.5: Akhir Permainan
4.	Natan Danuarta Ariel Wicaksana	13525143	Pada Bagian Program: - Pembuatan Rule dan Fakta Mainkan Kartu - Pembuatan Rule dan Fakta Tangkap - Pembuatan Rule dan Fakta UNI Pada Bagian Laporan: - Pembagian Tugas Laporan - BAB I: Deskripsi Masalah (Tujuan, Alur Permainan, dan Spesifikasi) - BAB II 2.2.1: mainkanKartu(NomorU

			rut). dan 2.2.4 uni(NomorUrut), (2.2.5) tangkap>NamaPemain). - BAB III: Inisialisasi Permainan, Siklus Turn Permainan, dan Fitur Khusus. - BAB IV: Pembagian dan Persentase Kerja
--	--	--	---

- Surat Pernyataan Penggunaan AI

SURAT PERNYATAAN PENGGUNAAN KECERDASAN BUATAN/AI

Dengan ini, ~~nama~~/kami*:

Theresia Estelina Ratu Udju/13525088 Christabelcyne Costan/13525141
 Nama/NIM : Edward Terrance Lie/13525127 Natan Danurta Ariel Wicaksono/13525143
 Fakultas/Sekolah : STEI-k / Teknik Informatika
 Kode Mata Kuliah : IF 1221
 Nama Mata Kuliah : Logika Komputasional
 Judul Tugas/Proposal : Praktikum (Tugas Besar) IF1221 - Logika Komputasional "UNI"
 Kelompok 6-25

Menyatakan bahwa ~~nama~~/kami* menggunakan ~~nama~~ AI/kecerdasan buatan dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan (level 1-5)
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya. Jelaskan secara singkat pemakaian dan tool yang digunakan: Membantu pemeriksaan kalimat serta koherensi, paragraf	Ya/ nama	3
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya. Jelaskan secara singkat pemakaian dan tool yang digunakan:	nama /Tidak*	1
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, <u>Zoom-Companion</u> , atau sejenisnya. Jelaskan secara singkat pemakaian dan tool yang digunakan: Membantu dalam menentukan kerangka / gambaran untuk laporan	Ya/ nama	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti <u>Craiyon</u> , DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya. Jelaskan secara singkat pemakaian dan tool yang digunakan:	nama /Tidak*	1
Lainnya Jelaskan secara singkat pemakaian dan tool yang digunakan: Penggunaan AI untuk menemukan masalah di dalam program (Debugging) serta membantu dalam pembelajaran terkait penggunaan term Prolog	Ya/ nama	3

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Kamis, 28 Mei 2026

Sumedang



Edward Terrance Lie 13525127
 Theresia Estelina Ratu Udju 13525088
 Christabelcyne Costan 13525141
 Natan Danurta Ariel Wicaksono 13525143

*coret yang tidak sesuai